

CSIS 2300 Introduction to Algorithms

Spring 2026 – Homework 7 Solutions

Instructions: You do not need to typeset your solutions; however, your submission must be a single PDF file. The assignment is due by **8:59pm on Tuesday, March 17**. Remember that you may submit as many times as you wish before the deadline (we'll only grade the last one), but late submissions will receive no credit – so submit early, submit often!

This assignment consists of problems numbered 1-6; be sure to look at the second page.

Let $h(k) = k \bmod 11$ be our hash function for the first two problems.

(1) Using chaining (open hashing), show the resulting hash table after placing values 40, 60, 37, 83, 42, and 18 into it. State the actual (not asymptotic) worst-case and average-case number of key comparisons needed for a successful search in this table. Show your work.

0		
1		
2		
3		
4	37	
5	60	
6	83	
7	40	18
8		
9	42	
10		

For a successful search, the worst case is just two comparisons (when trying to find 18). All other values require only one comparison, so the average case is $1\frac{1}{6}$.
 $(1 + 1 + 1 + 1 + 1 + 2)/6$.

(2) Same as (a) but using closed hashing with linear probing.

				37	60	83	40	18	42	
0	1	2	3	4	5	6	7	8	9	10

For a successful search, the worst case is just two comparisons (when trying to find 18). All other values require only one comparison, so the average case is $1\frac{1}{6}$. $(1 + 1 + 1 + 1 + 1 + 2)/6$. (Note that unsuccessful searches, though, have already gotten quite bad.)

(3) A student wants to build an open hash table (using chaining) that keeps each bucket in sorted order.

(a) What approach (i.e. algorithm) would you use to efficiently keep the keys within a bucket in sorted order and why?

If the buckets get fairly full, binary search is the clear winner for all three dictionary operations.

(b) Using your algorithm, what are the new asymptotic average-case runtimes for the insert, search, and delete operations for your hash table? Justify your answer.

Since the expected number of elements in each bucket is α and binary search is a logarithmic algorithm, the average-case runtime for all three operations should be $O(1 + \log \alpha)$. (Note: This makes sense – we incur a bit more cost on insertions for the benefit of a bit lower cost on search & delete.)

(4) Consider a hash table with $m = 1000$ buckets and a hash function $h(k) = \lfloor m(kA \bmod 1) \rfloor$ where $A = (\sqrt{5} - 1)/2$. (“mod 1” here is a commonplace abuse of notation – it means to only keep portion of the answer after the decimal point, discarding the whole number portion. Ex.: $6.8173 \bmod 1 = 0.8173$)

Compute the buckets into which the keys 61, 62, 63, 64, and 65 are hashed. (Show your work.) What do you observe about these locations?

$$h(61) = 700; h(62) = 318; h(63) = 936; h(64) = 554; h(65) = 172$$

The values are well spread out around the hash table. The gaps, however, are all the same – $h(k + 1) = h(k) + 618 \pmod{1000}$. This could be problematic.

(5) Suppose we are storing n items in an open hash table with m buckets. Let the universe of possible keys U have size $|U| > mn$. Prove that the worst-case runtime for the hash table's Search operation is $O(n)$.

Since any hash function must divide up the $|U|$ keys among the m buckets, the Pigeonhole Principle tells us that there is at least one bucket into which at least n of the keys will fall. It is therefore possible that all n keys fall into that specific bucket, which would mean that the hashtable's Search function would just become a linear search on a list of n items, which requires $O(n)$ time.

(6) (a) Compute the Jaccard similarity of the following sets:

$\{A,B,D,E,F,G,H,J,K,N,Q,R,V,W,X,Z\}$ and $\{B,C,D,F,G,I,J,L,M,N,O,P,R,S,T,U,W,X,Y,Z\}$.

The size of the union of the two sets is 26 (all letters). The intersection is $\{B,D,F,G,J,N,R,W,X,Z\}$, which has 10 elements. The Jaccard similarity is thus $10/26$.

(b) Prove that the probability that two sets produce the same minhash value is equal to the Jaccard similarity of the two sets.

A minhash will permute the columns of the membership matrix and then place each set into a hash bucket based upon the column containing the first 1.

If we examine the rows for two sets A & B, there are four types of columns:

1. Both sets have a 0 in the column. (The corresponding element is in neither set.)
2. Both sets have a 1 in the column. (The corresponding element is in both sets.)
3. Row A has a 1; Row B has a 0. (The corresponding element is in A and not in B.)
4. Row A has a 0; Row B has a 1. (The corresponding element is in B and not in A.)

A & B will hash to the same bucket iff, after permutation, we encounter a column of type 2 before we encounter a column of type 3 or 4. (Note that columns of type 1 have no effect.)

The number of columns of type 2 is precisely the size of $A \cap B$. The number of columns of types 2, 3, or 4 is the number of elements that exist in either set; i.e. $|A \cup B|$. Since we are going to permute the columns uniformly at random, the probability that, out of columns of types 2, 3, or 4, the first one we encounter is type 2 is $|A \cap B| \div |A \cup B|$ -- this is the Jaccard similarity.